Indian Institute of Technology (IIT-Bombay)

AUTUMN Semester, 2025

COMPUTER SCIENCE AND ENGINEERING

CS230: Digital Logic Design + Computer Architecture

Quiz II

Full Marks: 20

Time allowed: 1.0 hours

INSTRUCTIONS: Do not make any extra assumptions other than those stated in the questions. The marks will only be given if a precise, sound, and clearly written explanation is provided that is understandable by the examiners. Any extra unnecessary writing, verbal arguments based on that, and unnecessary crib will result in a reduction of 5 marks. Write answers inside the boxes.

1. **Just Chill!!** Who scored a century in the last ODI match against Australia?

    1. Virat Kohli
    2. Rohit Sharma
    3. Shubman Gill
    4. Shreyas Iyer

    (1 mark)

    Solution: Rohit Sharma

2. Choose the correct statement(s). One or multiple statement(s) can be correct. **You will only be given marks if you choose all the correct and no incorrect options.**

    1. Pipeline primarily improves the instruction throughput.
    2. Pipeline primarily improves the latency of each instruction.
    3. Cache hierarchy has no effect on the latency of program execution.
    4. Pipeline can afford higher clock frequency for a processor compared to a non-pipelined, single-cycle processor.

    (2 marks)

Solution: 1,4 Pipeline is mainly a throughput enhancement technique. The per instruction latency remains unchanged or even slightly degrades due to the pipeline registers (over a non-pipelined processor). Clock cycle time can drastically increase over a non-pipelined processor, as the critical path per stage is much smaller than the entire datapath of a non-pipelined processor. The cache hierarchy's sole purpose is to improve average memory access latency.

3. Choose the correct statement(s) for an in-order, 5-stage pipeline without any branch predictor. Ignore the effect of memory hierarchy (and related optimizations on memory hierarchy). One or multiple statement(s) can be correct. **You will only be given marks if you choose all the correct and no incorrect options.**

    1. Forwarding in the pipeline can solve all the data hazards without stalls.
    2. Structural hazards due to the register file can be resolved by allowing multiple ports, and writing in the first half of a clock cycle while reading in the second half.
    3. WAR and WAW dependencies are common in in-order processors.
    4. Hazard detection works by observing the pipeline registers.

    (2 marks)

Solution: 2,4. Forwarding cannot resolve data hazards without incurring stalls if, for example, a `load` instruction is immediately followed by a `add/sub` instruction. See slides for more details. Multiple ports and simultaneous read-write in one cycle are indeed ways to handle structural hazards due to register files. WAR and WAW would never happen for an in-order pipeline. Pipeline hazards are detected by checking the pipeline registers. See slides for details.

4. Suppose we have a pipelined processor for which we want to implement a simple branch predictor. For simplicity, let us only consider the conditional branches. Also, each conditional branch will have a separate predictor, and different conditional branches do not interfere with each other's predictor states. Consider the following piece of code:

```
for (i = 0; i < 100; i++) {
  for (j = 0; j < 10; j++) {
    ...
    B1: branch back to the for j loop
  }
  ...
  B2: branch back to the for i loop
}
```

B1 and B2 are the last instructions for the j loop and for the i loop, respectively. **Consider branch B1**. How many correct predictions will occur if we use the following prediction schemes?

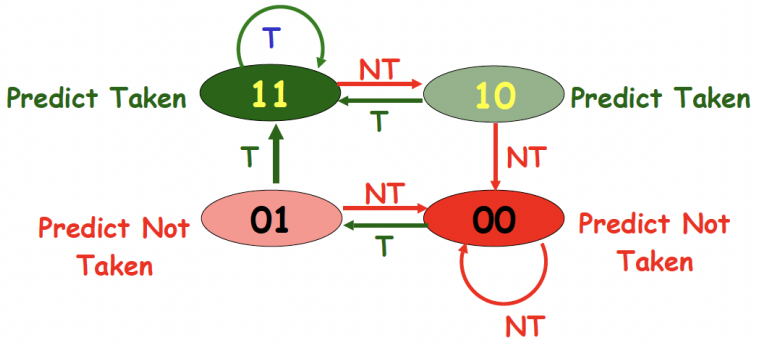

Figure 1: Question 4 (2-bit predictor)

1. 1-bit predictor initialized to **TAKEN** (T)
2. 2-bit predictor initialized to **10** (see Fig. 1)

For each case, you have to calculate the number of correct predictions with a proper explanation. Do not make any extra assumptions other than those stated in the question. (5 marks)

Solution:

1. For each execution of the outer (i) loop, the inner inner (j) loop runs for 10 times. Let us consider the first iteration of the outer loop. For a 1-bit predictor starting with T, the j loop will result in 9 **correct** (T), and one incorrect prediction at the end. The (last) incorrect prediction will change the state of the predictor to NT. Therefore, for the first iteration of the i-loop, there will be a total of 9 correct and 1 incorrect prediction. Now, from the second iteration onward, for the i loop, the predictor starts with NT state. Therefore, there will be a misprediction at the beginning, changing the state of the predictor from NT to T. Then, there will be correct predictions, except for the last one, which will again change the state back to NT. Therefore, from the second iteration onward for the outer loop, we shall see 2 mispredictions and 8 correct predictions. So, overall, the total number of correct predictions is given by $9 + 8 \times 99 = 801$.

2. Consider the first iteration of the i loop and the 10 iterations of the j loop within it. The start state is 10, so it will be predicted T, and the actual loop outcome is also T. So, no mispredictions here, and the state of the predictor will become 11. At last, there will be one misprediction (for NT), but the predictor will only go to state 10. For the next iteration of the i loop, j loop will start again from state 10. So, overall, there will be 1 misprediction per iteration. Overall, there

will be $100 \times 9 = 900$ correct predictions for B1.

5. Let us consider a 4KB direct-mapped cache with a miss rate of 0.06, where 67% of these misses are conflict misses. Dr. No suggested adding a victim cache with this direct-mapped cache, which removes 85% of the conflict misses in a program. **What is, i) AMAT without victim cache; ii) AMAT with victim cache; iii) the percentage improvement in the AMAT (average memory access time) due to the victim cache? Please write down every component of the AMAT calculation to get marks. Just writing answer without the proper calculation steps will not fetch marks.**

    1. Assume a simple, single-issue, 5-stage pipeline, in-order processor that blocks on every read and write until it completes.
    2. A hit in the main cache takes 1 cycle.
    3. Every miss in the main cache goes to the victim cache.
    4. For a miss in the main cache, 1 additional cycle is needed to access the victim cache.
    5. After missing in the main and victim caches, assume an additional penalty of 48 cycles to get the data from memory.

    (5 marks)

Solution: $AMAT_{noVC} = 1$ (cache hit) $+ 0.06 * 48$ (miss rate * miss penalty) $= 3.88$ cycles.
$AMAT_{VC} = 1$ (cache hit) $+ 0.06 * \{(0.67 * 0.85 * 1)$ (victim cache hit) $+ (1 - (0.67 * 0.85)) *$ 49 (victim cache miss)$\} = 2.29984$ cycles.
Percentage Improvement for AMAT: $\frac{3.88 - 2.29984}{3.88} * 100 = 40.7\%$.

6. This question is based on your grandfather's computer. Your grandfather has a new dual Pentium processor (2 processor cores), and you have been tasked with optimizing your software for this processor. You will run two applications (individually, not simultaneously) on this dual Pentium, but the resource requirements are not equal. The first application (App1) needs 80% of the resources, and the other (App2) only 20% of the resources. By x% *resource requirement* for an application, we mean that x% of the *overall system time* is spent on running that application. Assume that the speedup is to be calculated with respect to a baseline single-core processor.

    1. Given that 40% of the first application is parallelizable, how much speedup would you achieve with that application if run in isolation? (0.5 marks)
    2. Given that 99% of the second application is parallelizable, how much speedup would this application observe if run in isolation? (0.5 marks)

3. Given that 40% of the first application is parallelizable, how much overall *system speedup* would you observe if you parallelized it? Assume App2 is not running, but other applications are running with App1, so that all the resources are occupied. (2 marks)
4. Given that 99% of the second application is parallelizable, how much overall system speedup would you get? Assume App1 is not running, but other applications are running with App2, so that all the resources are occupied. (2 marks)

Solution:

1. $Speedup = \frac{1}{0.6+\frac{0.4}{2}} = 1.25X$
2. $Speedup = \frac{1}{0.01+\frac{0.99}{2}} = 1.98X$
3. $Speedup = \frac{1}{0.2+0.8*(0.6+\frac{0.4}{2})} = 1.19X$
4. $Speedup = \frac{1}{0.8+0.2*(0.01+\frac{0.99}{2})} = 1.11X$